

MTech Rocks: Optimizing the ext4/JBD2 Consistency Mechanism on Linux 6.1.4

Milan Roy (241110042) Sahil Basia (241110061) Shrey Sharma (251110068)
Suyamoon Pathak (241110091)

CS614 Linux Kernel Programming
Indian Institute of Technology Kanpur

April 2026

Abstract

This report describes the work of the **MTech Rocks** group on the ext4/JBD2 consistency mechanism on Linux 6.1.4. Milan benchmarked the baseline cost of each ext4 journaling mode and identified where the journal hurts performance most; this characterization gave the rest of the group concrete directions to chase. Suyamoon then closed two fast-commit fallback paths with small kernel patches: `EXT4_FC_REASON_XATTR` (108 lines, 64× fewer JBD2 full commits on a setxattr workload, 27% wall-time gain on VM and 48% on bare-metal) and `EXT4_FC_REASON_FALLOC_RANGE` (46 lines, 62.5× fewer commits on collapse/insert range, 23–26% on VM and 42–44% on bare-metal). Sahil built a kernel module that uses `kretprobes` to first measure that ~78% of fsync time is spent inside `jbd2_log_wait_commit`, then replaces that function with a lockless CAS version (+40% / +54% / +7% IOPS at 16 threads on three workloads, no streaming regression). Shrey built a kernel module that replaces JBD2’s strict ring-buffer journal allocator with a free-block bitmap, cutting P99 latency on a WAL workload from 0.41 ms to 0.05 ms in `ordered` mode at the same write amplification factor.

Contents

1	Introduction	3
2	Workload Characterization (Milan)	3
2.1	Why characterization first	3
2.2	Headline numbers (WAL, 1 thread, 100 MB, fsync per write, 4 KiB)	3
2.3	What this told the group	4
2.4	Concurrent workload	4
2.5	How to reproduce	4
3	Suyamoon’s Four Candidates: Overview	4
4	Candidates 1 and 2: Did Not Measure	5
4.1	C1: fast-commit barrier deferral	5
4.2	C2: async bitmap prefetch completion	5
5	Candidate 3: Fast Commit Support for Inline xattrs	6
5.1	Intuition	6
5.2	Code analysis	6
5.3	Design and implementation	6

5.4	Correctness	7
5.5	Results	7
6	Candidate 4: Fast Commit Support for fallocate Range Operations	7
6.1	Intuition	7
6.2	Characterization first	7
6.3	Code analysis	8
6.4	Design and implementation	8
6.5	Correctness	9
6.6	Results	9
7	Candidate 5: Lockless CAS for jbd2_log_wait_commit (Sahil)	9
7.1	Analyzing the lock latency of JBD2	10
7.2	Identifying lock contention time	10
7.3	Mitigating lock contention with CAS synchronization	11
7.4	Benchmark evaluation and thread scaling	12
7.5	Reproduction	13
8	Candidate 6: Fragmented Journal Map (FJM) for ext4 (Shrey)	13
8.1	Conventional vs. fragmented journaling	13
8.2	Maintaining the journal and circular transaction sequence	13
8.3	Fragmented block occupation	14
8.4	Journal space efficiency	14
8.5	Redundancy of credit-based reservation	14
8.6	Experiments	14
8.7	Reproduction	15
9	Conclusion	15
A	Artifact Evaluation	17
A.1	Artifact directory structure	17
A.2	Setup	17
A.3	Features supported	18
A.4	Assumptions and unsupported	18
A.5	Getting started (within 30 minutes)	18
A.6	Detailed evaluation	18

1 Introduction

This project asks: *where does ext4’s consistency mechanism spend its time on Linux 6.1.4, and what can be done about it?* The mechanism in question is JBD2 (the journaling layer that keeps the filesystem crash-consistent) and its fast-commit extension that was added in Linux 5.10 to make `fsync()` cheaper. Any change must keep crash recovery working and must be backed by measurement.

The four members of **MTech Rocks** divided the work like this:

- **Milan** measured the baseline cost of each ext4 journaling mode (`ordered`, `journal`, `writeback`, `nojournal`) on representative WAL and concurrent workloads. Section 2. The numbers told the rest of the group where to look for optimizations.
- **Suyamoon** worked on two coverage gaps in fast commit. Sections 3 (overview of four candidates), 4 (C1, C2 – did not measure), 5 (C3, inline xattrs), and 6 (C4, fallocation range).
- **Sahil** worked on the global `j_state_lock` taken inside `jbd2_log_wait_commit` by every `fsync` waiter. Section 7 (Candidate 5).
- **Shrey** worked on the journal layout itself, replacing the strict ring-buffer allocator with a free-block bitmap. Section 8 (Candidate 6).

Each section is written by the member who did the work and starts with what was built and how to reproduce it.

2 Workload Characterization (Milan)

Author: Milan Roy (241110042).

2.1 Why characterization first

Before changing the kernel it is worth knowing where the time actually goes. The two harnesses in `milan/benchmarks/` answer that on a real NVMe partition for two complementary workloads:

- `wal_workload_no_fc/` – single-threaded sync-heavy fio with `fdatasync` after every write, parameterised by block size $\{4, 8, 16, 32, 64\}$ KiB. This is the classic database WAL pattern.
- `combined_workload_no_fc/` – the WAL pattern run *concurrently* with a bulk 1 MiB sequential writer. Both fios are bounded to the same wall time so they contend for journal resources for the full window.

Each harness sweeps four ext4 modes (`ordered`, `journal`, `writeback`, `nojournal`), runs five repetitions per mode, and reports mean $\pm$ stddev. `bpftime` is attached to the JBD2 tracepoints (`jbd2_journal*_callback`, `jbd2_handle_*`) for the full run. Fast commit is intentionally off so the numbers reflect the baseline JBD2 cost.

2.2 Headline numbers (WAL, 1 thread, 100 MB, fsync per write, 4 KiB)

Mode	IOPS	BW	p50 lat	p99 lat	IOPS penalty
ordered	185 $\pm$ 4	0.72 MB/s	5.12 ms	8.70 ms	51.3%
journal	179 $\pm$ 3	0.70 MB/s	5.25 ms	10.13 ms	52.9%
writeback	185 $\pm$ 2	0.72 MB/s	5.15 ms	8.94 ms	51.4%
nojournal	381 $\pm$ 3	1.49 MB/s	2.41 ms	7.69 ms	baseline

2.3 What this told the group

Three findings shaped the optimization choices in the rest of the report:

1. **Journaling halves IOPS regardless of mode.** The difference between `ordered`, `journal`, and `writeback` is within noise on this workload. The cost of journaling is the cost of *having a journal*, not of any particular mode.
2. **The cost is in the commit, not in the write.** Latency split shows **99.6% of total per-op latency is the `fdatsync` portion** across all journaled modes. This is what convinced the group that the productive optimizations would target the commit path (Suyamoon’s two patches) and the wait-on-commit path (Sahil’s module), not the write path.
3. **Tail latency is variance-dominated.** p99.99 in `ordered` mode is 159 ± 80 ms (50% relative stddev). Single-run reports of tail latency are not trustworthy here; the N-run averaging discipline used by the rest of the work below is necessary, not optional.

2.4 Concurrent workload

The combined harness runs the WAL fio job and a 1 MiB sequential fio job in the same window. The qualitative result is that the sequential workload’s bandwidth is mostly mode-insensitive (SSD-bound), but the WAL workload’s tail latency degrades sharply under contention because of journal resource pressure. The full per-mode tables are committed alongside the scripts as `*.txt` files in `milan/benchmarks/combined_workload_no_fc/`.

2.5 How to reproduce

```
cd "MTech Rocks/milan/benchmarks/wal_workload_no_fc"
sudo bash run_repeated.sh 5      # ~50 minutes

cd ../combined_workload_no_fc
sudo bash run_repeated.sh 3      # ~25 minutes
```

The default device is `/dev/nvme0n1p6` mounted at `/media/milan-roy/test_ext4`; edit the `DEVICE` and `MOUNT_POINT` variables at the top of the `run*_analysis.sh` scripts for a different machine.

3 Suyamoon’s Four Candidates: Overview

Sections 3–6 author: Suyamoon Pathak (241110091).

Milan’s measurements pointed at the commit path. The natural question was: which specific commit can we make cheaper or skip entirely?

The fast-commit feature already does this for many operations: it keeps a small reserved region of the journal and writes a compact record per modified inode instead of running a full JBD2 commit. But fast commit gives up on operations it does not know how to log. When `ext4` hits one of those it calls `ext4_fc_mark_ineligible(_,REASON,_)` and falls back to the full commit path. The original FastCommit paper [4] lists ten such reasons, including `XATTR`, `FALLOC_RANGE`, `CROSS_RENAME`, and `RENAME_DIR`. Two of those are what the patches in Sections 5 and 6 close.

Four candidates were tried in order; only the last two worked. Section 4 covers C1 and C2 briefly because the lessons they taught shaped how C3 and C4 were chosen.

4 Candidates 1 and 2: Did Not Measure

Both candidates were implementations of long-standing TODO comments in the kernel. Both were correct (built, booted, mounted, no dmesg errors), but neither produced a measurable speedup on our workloads.

4.1 C1: fast-commit barrier deferral

The TODO sits at `fs/jbd2/commit.c` around line 454:

```
/*
 * TODO: by blocking fast commits here, we are increasing
 * fsync() latency slightly. Strictly speaking, we don't need
 * to block fast commits until the transaction enters T_FLUSH
 * state. So an optimization is possible where we block new fast
 * commits here and wait for existing ones to complete
 * just before we enter T_FLUSH. That way, the existing fast
 * commits and this full commit can proceed parallely.
 */
```

We moved the drain loop that waits for in-flight fast commits from before `T_LOCKED` to just before `T_FLUSH`, along with the `journal->j_fc_off = 0` reset that depends on it. On our fio randwrite+fsync workload the patched average commit time was 6661 μ s versus 6590 μ s on stock, a 1% delta in the wrong direction. The `/proc/fs/jbd2/loopX-8/info` counter showed the original barrier almost never fired in this workload because `T_LOCKED` already takes well under a millisecond. An earlier “57% improvement” result turned out to be a setup artifact: the stock benchmark had run for 5 MB while the patched benchmark had run for 60 seconds, and the test filesystem was not formatted with `-O fast_commit` at all (so the drain loop was never executed). After fixing both, no improvement remained. Patch and postmortem are at `suyamoon/jbd2-fc-barrier-defer.patch` and `suyamoon/CANDIDATE1_postmortem.md`.

4.2 C2: async bitmap prefetch completion

The TODO sits at `fs/ext4/mballoc.c:2564`:

```
/*
 * TODO: We should actually kick off the buddy bitmap setup in a work
 * queue when the buffer I/O is completed, so that we don't block
 * waiting for the block allocation bitmap read to finish when
 * ext4_mb_prefetch_fini is called from ext4_mb_regular_allocator().
 */
```

We added a per-superblock workqueue and slab cache; in `ext4_mb_prefetch_fini`, if the bitmap is already up-to-date the original synchronous init is used, otherwise a work item is queued and the caller returns. After two correctness fixes caught by code review (a `noinline_for_stack` attribute that we had silently stripped, and an OOM-path leak of `s_buddy_cache`), the patch built cleanly. The problem was again measurement: in our microbenchmark the cold-init path was hit on <1% of allocations (the bitmaps were already in memory after the first few iterations), and our fio throughput benchmark had a 70% coefficient of variation across repeats on the same kernel. We did not invest further. Patch and postmortem are at `suyamoon/mballoc-async-prefetch.patch` and `suyamoon/CANDIDATE2_postmortem.md`.

Lesson. A patch is only as useful as the fraction of operations that exercise the path it changes. After C1 and C2 we adopted a hard rule: only ship a patch if a microbench can show that the slow path it removes is hit on a substantial fraction of the target operation.

5 Candidate 3: Fast Commit Support for Inline xattrs

5.1 Intuition

Every `setxattr` or `removexattr` call on a fast-commit-enabled ext4 filesystem forces a full JBD2 commit. The cost is paid on every SELinux relabel, every POSIX ACL change, and every short `user.*` attribute write, which makes the slow path effectively 100% hit on xattr-touching workloads. The ATC 2024 paper [4] flags this gap but provides no patch.

5.2 Code analysis

The fallback is fired by an unconditional call at the end of `ext4_xattr_set_handle` in `fs/ext4/xattr.c` (line 2422):

```
ext4_fc_mark_ineligible(inode->i_sb, EXT4_FC_REASON_XATTR, handle);
```

A second `mark_ineligible` call sits at line 2500 inside the wrapper `ext4_xattr_set`, after `ext4_journal_stop`, with `handle==NULL`. With `handle` null this call marks the running JBD2 transaction ineligible, which would suppress fast commit on any later `fsync` sharing that transaction. Missing this second call would have made the patch a no-op for end users.

ext4 stores xattrs in three places: *inline* (in the inode body, between `EXT4_GOOD_OLD_INODE_SIZE` + `i_extra_isize` and the end of the inode), in a separate *block* (pointed to by `i_file_acl`), or in dedicated *EA inodes*. We targeted the inline case because it covers nearly all real-world xattr operations: SELinux labels and POSIX ACLs are short and fit inline.

5.3 Design and implementation

Two changes, both small.

1. **Expand the FC inode log.** `ext4_fc_write_inode` in `fs/ext4/fast_commit.c:863` currently logs the inode's bytes up to `EXT4_GOOD_OLD_INODE_SIZE` + `i_extra_isize`. We extend it to log up to `EXT4_INODE_SIZE` when the inode has inline xattrs (detected via the existing `EXT4_STATE_XATTR` flag). Replay needs no change: `ext4_fc_replay_inode` already `memcpy`s `fc_len` bytes, and the validator already accepts records up to `s_inode_size`.
2. **Skip the ineligibility call when safe.** In `ext4_xattr_set_handle`, add a local `bool touched_block` that is set to true on every path that calls `ext4_xattr_block_set` or sets `i_in_inode = 1` (the EA-inode path). Replace the unconditional `mark_ineligible` at line 2422 with:

```
if (error || touched_block || !ext4_has_feature_xattr(inode->i_sb))
    ext4_fc_mark_ineligible(inode->i_sb,
                           EXT4_FC_REASON_XATTR, handle);
```

and delete the wrapper's redundant call at line 2500. The third condition handles the very first xattr on a fresh filesystem (the superblock xattr feature bit has just been set in memory but not yet persisted; if we fast-committed and the machine crashed, replay would not see the bit).

The full patch is 108 lines (`suyamoon/fc-inline-xattr.patch`): +9/-0 in `fast_commit.c` (the new conditional with its comment), and +27/-4 in `xattr.c` (the `touched_block` declaration, five assignments to it on each block-touching path, the new conditional at line 2422, the removal of the redundant call at line 2500, and explanatory comments at both sites).

5.4 Correctness

- Three crash-recovery scripts (`c3_crash_test_{a,b,c}.sh`). Test A: 100 inline xattrs, lazy unmount, remount, count (**PASS**, 100/100). Test B: 100 set, 50 even removed, lazy unmount, remount, verify exactly the 50 odd ones remain (**PASS**). Test C: a 668-byte xattr value (well over the inline limit so the xattr block path is used), lazy unmount, remount, full value recovered (**PASS**, confirming the fallback branch still runs the full commit when needed).
- xfstests subset `generic/{062,118,300,337,454,455,473,482}` + `ext4/032`. Of the nine tests, three skip due to missing features (reflink for `generic/118`, `LOGWRITES_DEV` for `generic/455` and `482`). Of the six that ran, five pass and `generic/473` fails identically on stock and patched (a pre-existing 6.1.4 issue, not a regression). Logs at `suyamoon/xfstests_c3/`.

5.5 Results

VM (VirtualBox, 11th Gen Intel Core i7-11800H, 4 cores, 5.7 GiB RAM; 256 MB loop fs with `-O fast_commit`; 5000 `fsetxattr+fsync` on a single inline-sized value).

Metric	Stock	Patched	Change
Wall time	31,102 ms	22,734 ms	−26.9%
JBD2 full commits	5,000	78	−98.4%

Bare-metal (11th Gen Intel Core i3-1115G4, 4 cores, 7.5 GiB RAM, real SATA SSD; 3 repeats).

Metric	Stock	Patched	Change
Wall time (mean)	57,627 ms	30,089 ms	−47.8%
JBD2 full commits	5,000	78	−98.4%

The transaction count reduction (5000 to 78, $\sim 64\times$) is identical between VM and bare-metal: this is the architectural claim of the patch and it does not depend on the hardware. The wall-time gain is larger on bare-metal (48% versus 27%) because on a real SSD the fixed VFS and syscall overhead per `fsync` is a smaller share of the total, leaving more room for the journal-side savings to show through.

A non-xattr regression check (`fio randwrite+fsync`, 4 jobs, 30 s) gave 528 IOPS patched versus 530 stock, well within noise.

6 Candidate 4: Fast Commit Support for fallocate Range Operations

6.1 Intuition

After C3 we asked which other `ext4_fc_mark_ineligible` fallback has the same shape: a 100% hit rate on a well-defined operation class, a small expected patch, and a microbench that cleanly distinguishes patched from unpatched behavior.

6.2 Characterization first

For C4 we measured before writing kernel code. Three candidate microbenchmarks were run on the C3 kernel: fallocate COLLAPSE/INSERT range; large (4 KB) `setxattr` values that spill to the xattr block (the case C3 does not cover); and many-thread concurrent `fsync` (the CJFS-style compound flush target). Three runs each, $N = 1000$ ops (or 100 per thread):

Workload	ms/op	tx/op	signal
fallocate COLLAPSE_RANGE	7.03	1.001	strong
fallocate INSERT_RANGE	7.01	1.001	strong
xattr block (4 KB value)	7.23	1.001	strong but narrow
concurrent fsync $T = 16$	1.21	0.003	weak

Fallocate COLLAPSE/INSERT showed the same per-op signal as C3's xattr case. The xattr-block path showed equal per-op cost but covers a minority of `setxattr` calls (typical SELinux/ACL values are short and already handled by C3). Concurrent fsync was already well-coalesced by JBD2's group commit (tx/op falls from 0.020 at $T = 1$ to 0.003 at $T = 16$), so a CJFS-style port would not help much on 6.1.4. Fallocate became C4.

6.3 Code analysis

Two call sites in `fs/ext4/extents.c` mark fallocate range operations as ineligible, both right after `ext4_journal_start`:

```
/* line 5363, inside ext4_collapse_range */
ext4_fc_mark_ineligible(sb, EXT4_FC_REASON_FALLOC_RANGE, handle);

/* line 5508, inside ext4_insert_range */
ext4_fc_mark_ineligible(sb, EXT4_FC_REASON_FALLOC_RANGE, handle);
```

Crucially, ext4 already has fast-commit tags for the underlying extent-level changes (FC_TAG_ADD_RANGE, FC_TAG_DEL_RANGE, emitted by `ext4_fc_write_inode_data`), and the per-inode FC_TAG_INODE tag carries the new `i_size`. The fallback exists only because the collapse/insert paths never call `ext4_fc_track_range` on the logical blocks they modify, so the FC commit walker does not know to include the shifted region.

6.4 Design and implementation

Replace each of the two `mark_ineligible` calls with a call to `ext4_fc_track_range` over the affected logical-block range, computed at the point where the journal handle is open:

```
/* collapse path: track punch_start to current EOF */
ext4_fc_track_range(handle, inode, punch_start,
    ((inode->i_size + inode->i_sb->s_blocksize - 1) >>
    EXT4_BLOCK_SIZE_BITS(inode->i_sb)) - 1);

/* insert path: track offset_lblk to post-grow EOF */
ext4_fc_track_range(handle, inode, offset_lblk,
    ((inode->i_size + len + inode->i_sb->s_blocksize - 1) >>
    EXT4_BLOCK_SIZE_BITS(inode->i_sb)) - 1);
```

At fsync time, `ext4_fc_write_inode_data` walks the post-shift extent tree over the tracked range and emits FC_TAG_ADD_RANGE for each shifted extent at its new position plus FC_TAG_DEL_RANGE for any resulting hole. FC_TAG_INODE is emitted by the `ext4_mark_inode_dirty` call later in each function and carries the new `i_size`.

Replay correctness. The non-obvious part is that `ext4_fc_replay_del_range` frees the physical blocks of the range it removes via `ext4_mb_mark_bb(..., 0)`. In an insert (where the physical data does not move, only the logical position shifts) DEL_RANGE replay transiently marks blocks free and ADD_RANGE replay re-binds them. The intermediate bitmap state is wrong; the final state is reconciled by the pre-existing `ext4_fc_set_bitmaps_and_counters` pass, which runs at the end of replay and re-marks all currently-reachable blocks as used.

The full patch is 46 lines (`suyamoon/fc-fallocate-range.patch`), two hunks in `fs/ext4/extents.c`. No change to `fast_commit.c`, no new tag types, no on-disk format change.

6.5 Correctness

- Three crash-recovery scripts (`c4_crash_test_{a,b,c}.sh`). Each writes a distinct-byte-pattern file, performs the operation, fsyncs, lazy-unmounts to simulate a crash, remounts, and compares the resulting file’s md5 and size to the expected post-operation state computed in pure Python. Test A: collapse 32 blocks (**PASS**). Test B: insert 16 blocks at offset 16 (**PASS**). Test C: three interleaved collapse/insert ops (**PASS**).
- Same xfstests subset as C3. Five of the six runnable tests pass; `generic/473` again fails identically to stock and to the C3 kernel.

6.6 Results

VM (1000 ops + per-op fsync, three runs per mode). The C3 kernel is the baseline because C3 does not change any fallocate code; on this path C3 and stock 6.1.4 are architecturally identical.

Metric	Baseline (C3)	Patched (C4)	Change
<i>COLLAPSE_RANGE</i>			
Wall time (mean)	7,029 ms	5,423 ms	−22.8%
JBD2 full commits	1,001	16	−98.4%
<i>INSERT_RANGE</i>			
Wall time (mean)	7,006 ms	5,189 ms	−25.9%
JBD2 full commits	1,001	16	−98.4%

Bare-metal (i3-1115G4, real SATA SSD; 3 repeats).

Metric	Baseline (C3)	Patched (C3+C4)	Change
<i>COLLAPSE_RANGE</i>			
Wall time (mean)	12,445 ms	7,032 ms	−43.5%
JBD2 full commits	1,001	16	−98.4%
<i>INSERT_RANGE</i>			
Wall time (mean)	11,539 ms	6,680 ms	−42.1%
JBD2 full commits	1,001	16	−98.4%

The transaction-count reduction is byte-identical to the VM (1001 → 16 in both modes). The wall-time gain is roughly double on bare-metal (43–44% vs 23–26%), matching the C3 pattern. Patched-run variance is small: 0.5% CV for COLLAPSE, 2% for INSERT.

7 Candidate 5: Lockless CAS for `jbd2_log_wait_commit` (Sahil)

Author: Sahil Basia (241110061).

7.1 Analyzing the lock latency of JBD2

High-concurrency applications rely on the `fsync()` system call to safely flush in-memory data to the physical disk. In the ext4 filesystem this is handled by the JBD2 (Journaling Block Device 2) layer. During a transaction, any application threads that request a synchronous flush are put to sleep on the `j_wait_done_commit` queue. A separate, single background daemon thread is responsible for actually writing the data to the disk.

Our architectural review of the kernel code identified a major scaling bottleneck within the `jbd2_log_wait_commit()` function. To manage the sleeping threads safely, the native kernel places a global read-write lock (`j_state_lock`) around the code that checks if the commit is finished.

When the background daemon finishes writing to the disk, it sends a wakeup signal to all the sleeping application threads at the exact same time. The problem occurs immediately after they wake up: every single thread must acquire the exact same `j_state_lock` to verify if their specific data was committed. Because only a limited number of operations can modify the lock code at once, the threads spend a massive amount of CPU time simply waiting in a queue to acquire the lock, severely delaying their return to the user application.

Listing 1: The Native ext4 Implementation (Problematic)

```
int jbd2_log_wait_commit(journal_t *journal, tid_t tid)
{
    int err = 0;

    /* Severe scaling bottleneck: global RW lock */
    read_lock(&journal->j_state_lock);

    while (tid_gt(tid, journal->j_commit_request)) {
        journal->j_commit_request = tid;
        wake_up(&journal->j_wait_commit);
    }
    read_unlock(&journal->j_state_lock);

    wait_event(journal->j_wait_done_commit,
               !tid_gt(tid, journal->j_commit_sequence));

    return err;
}
```

7.2 Identifying lock contention time

To analyze this wait-time overhead without artificially slowing down the system using heavy sampling tools, we built a lightweight, dynamic kernel module utilizing the `kretprobes` API. Rather than modifying the filesystem directly, this module passively hooks into the entry and return boundaries of both the `fsync()` system paths and the `jbd2_log_wait_commit()` function.

Internal to the module, atomic variables continually aggregate the total nanoseconds spent executing the overarching file synchronization against the absolute time bounded specifically inside the wait-queue lock. Upon cleanly unloading the module using the `rmmod` command, our custom exit handler automatically calculates the proportion of wasted CPU time and dumps a synthesized statistical summary directly into the Linux kernel ring buffer.

By analyzing the resulting kernel `dmesg` logs after running multiple threads simultaneously, the output consistently mapped the latency bottleneck. Reading the custom latency tracker empirically proved that simply waiting to acquire the `j_state_lock` was severely restricting the filesystem's ability to process parallel system operations:

Listing 2: Sample `dmesg` output from the trace module on `rmmod`.

```

[ 4213.153921] jbd2_trace: =====
[ 4213.153922] jbd2_trace: JBD2 Trace Module Unloaded.
[ 4213.153924] jbd2_trace: Module State: Baseline (opt=0)
[ 4213.153925] jbd2_trace: -----
[ 4213.153928] jbd2_trace: Total Fsyncs Tracked      : 24210
[ 4213.153930] jbd2_trace: Total Workload Time       : 16503859230 ns
[ 4213.153931] jbd2_trace: Time Spent In JBD2 Lock: 12928502010 ns
[ 4213.153933] jbd2_trace: Target lock latency hit: 78% overhead
[ 4213.153935] jbd2_trace: =====

```

7.3 Mitigating lock contention with CAS synchronization

To solve this lock waiting issue without compromising ext4's data safety rules, we replaced the native `jb2_log_wait_commit` function with a scalable wait structure. This optimization removes the need for every thread to acquire the lock by using atomic Compare-And-Swap (CAS) instructions:

Listing 3: Optimized CAS lockless implementation

```

int lockless_log_wait_commit(journal_t *journal, tid_t tid)
{
    int err = 0;

    /* 1. Volatile read avoiding lock access entirely */
    if (!tid_gt(tid, READ_ONCE(journal->j_commit_sequence)))
        goto out;

    /* 2. Lockless wakeup via CAS spin-gate */
    if (tid_gt(tid, READ_ONCE(journal->j_commit_request))) {
        if (atomic_cmpxchg(&wakeup_cas, 0, 1) == 0) {
            read_lock(&journal->j_state_lock);
            if (tid_gt(tid, journal->j_commit_request))
                wake_up(&journal->j_wait_commit);
            read_unlock(&journal->j_state_lock);
            atomic_set(&wakeup_cas, 0); // Open gate
        }
    }

    /* 3. Sleep safely without re-locking */
    wait_event(journal->j_wait_done_commit,
        !tid_gt(tid, READ_ONCE(journal->j_commit_sequence)));

    /* 4. Enforce strict disk sync barrier */
    smp_rmb();
out:
    return err;
}

```

1. **Direct memory checking.** Instead of acquiring a heavy software lock just to check if a transaction is complete, waiting threads now use `READ_ONCE()`. This forces the C compiler to fetch the transaction status directly from the computer's main RAM, specifically preventing it from relying on outdated local CPU caches. If this direct read confirms the background daemon has already finished writing the data to the disk, the thread instantly exits the function without ever needing to touch a lock.
2. **Atomic wakeup.** When multiple threads realize that the background daemon needs to be actively signaled to flush the disk, they logically race to wake it up. Normally, every thread queues up to grab the lock to do this safely. We replaced this queue with a single atomic hardware instruction (`atomic_cmpxchg()`) which acts as a rigid ticket system. The very

first thread to arrive “takes the ticket” (changing a 0 to a 1 in a single, uninterrupted CPU cycle) and is permitted to acquire the lock to wake the daemon. Every thread that arrives a nanosecond later sees that the ticket is already taken. Because they know the first thread is already handling the wakeup signal, they safely bypass the lock entirely and immediately go to sleep.

3. **Ensuring correct execution order.** Removing the primary lock created a dangerous side effect. Lock functions implicitly act as heavy anchors that force the computer’s CPU to execute memory operations in a strict, sequential order. Without the lock, highly optimized CPUs might try to run operations out of order to save time, meaning a thread could return control back to the user’s database application *before* the system truly verified the disk operations were finished. To prevent this data corruption, we inserted an explicit hardware fence instruction (`smp_rmb()`) exactly at the point where threads wake up. This forces the physical CPU hardware to pause and guarantee that all disk operations are strictly finalized before allowing the software to proceed any further.

7.4 Benchmark evaluation and thread scaling

To evaluate the success of this optimization, we constructed an automated, multi-dimensional benchmarking suite. Tests were scaled across concurrency levels from 1 up to 16 threads, targeting both ext4 `data=ordered` and `data=writeback` journaling modes. We deliberately avoided the `data=journal` mode, as full data-journaling inherently imposes a 50% I/O write penalty. This penalty immediately bottlenecks the physical SSD hardware long before any CPU lock contention can natively manifest.

The evaluation matrix used four distinct testing scenarios:

- **Wait-queue contention (fio 4K random write).** By forcing dozens of threads to write extremely small 4K blocks and demanding an `fsync()` after every single operation, this benchmark intentionally triggers the exact `j_state_lock` queueing problem.
- **Database simulation (sysbench OLTP fileio).** Mirrors the execution pattern of production relational databases (continuous random updates across large logical file structures).
- **Metadata stress (fio create/unlink).** ext4 primarily uses JBD2 to safeguard metadata. By forcing threads to rapidly allocate (create) and destroy (unlink) thousands of files we saturate the filesystem’s structural inode and block-bitmap trees.
- **Data-intensive streaming (fio 10 MB sequential).** This boundary test establishes our “no-regression” constraint. Heavy 10 MB sequential data dumps instantly bypass the CPU overhead limits and bottleneck the physical SSD hardware.

Workload (16 threads)	Baseline	CAS Opt.	Change
Wait-queue contention	14,200 IOPS	19,850 IOPS	+40%
Metadata stress	8,500 IOPS	13,100 IOPS	+54%
Database OLTP	21,400 IOPS	22,900 IOPS	+7%
Sequential streaming	SSD-bound	SSD-bound	no regression

While the latency of single-threaded tasks remained identical to the baseline, the dynamically optimized module showed massive improvements as the active thread count scaled. Because the CAS gate prevented the threads from queueing for the `j_state_lock`, internal wait times dropped significantly. This optimization structurally allowed the filesystem’s processing throughput to scale upwards alongside the number of active hardware threads, without compromising disk integrity.

7.5 Reproduction

```
cd "MTech Rocks/sahil/module"
sudo apt install -y linux-headers-$(uname -r)
make clean && make                # builds jbd2_trace.ko

cd ../benchmarks
sudo ./validation.sh              # crash-safety smoke (fsck.ext4 -n)
sudo ./dmesg_sampler.sh           # baseline lock-overhead %
sudo ./run_evals.sh               # full scaling matrix (~30-60 min)
python3 plot_results.py
```

8 Candidate 6: Fragmented Journal Map (FJM) for ext4 (Shrey)

Author: Shrey Sharma (251110068).

8.1 Conventional vs. fragmented journaling

Conventional JBD2 journaling relies on a sequential pointer logic (`j_head` and `j_tail`) flowing through an on-disk ring buffer. When a transaction commits, its description, payload, and commit records must be written contiguously. While theoretically simple to recover, this design suffers from acute internal fragmentation: if a multi-block transaction does not fit before the end of the journal boundary, it triggers complex wraparound edge-cases or forces the filesystem to prematurely checkpoint and block application I/O.

In contrast, the **Fragmented Journal Map (FJM)** is a hybrid metadata allocation strategy. It intercepts block mapping requests directly inside JBD2's write loop via a custom callback (`journal->j_alloc_block_callback`). Instead of forcing consecutive physical log blocks, FJM fractures the transaction over any available logical block in the journal using an in-memory allocation bitmap, allowing ext4 to decouple logical transaction sequences from physical journal contiguity.

8.2 Maintaining the journal and circular transaction sequence

Despite scattering the data across the physical journal space, the FJM tracks transaction lifecycle states and maintains sequence ordering.

In-memory tracking. When `__ext4_journal_start_sb` is called, the FJM records a new `fjmap_txn` linked directly to the running transaction's ID (`t_tid`). Upon journal block requests, the FJM allocator uses a bitwise `find_first_zero_bit()` search to secure a valid target block, updates its global bitmap to mark the block in-use, and maps the block's physical location back to the parent transaction array.

Circular enforcement via on-disk indexes. To handle crashes, transaction integrity is stored via an **FJM index block**. Although data sits fragmented across the device, the FJM intercepts the JBD2 commit phase (`ext4_fjmap_commit_txn`) and statically writes a contiguous reference table containing an `fjmap_ondisk_hdr` structure. This index maps sequence order (`seq`), logical transaction ID (`tid`), and physical `journal_blk` into a safe enclave (typically `journal_blk=1`). The circularity is preserved dynamically; older transactions are checkpointed and freed, clearing bits asynchronously in the bitmap array, while newer transactions seamlessly inherit those fragmented gaps.

8.3 Fragmented block occupation

When a transaction occupies fragmented blocks, it bypasses the `journal->j_head++` pointer arithmetic. Instead:

1. ext4 starts a transaction.
2. JBD2 aggregates buffers and calls `jbd2_journal_next_log_block()`.
3. The FJM override executes. Suppose block 5 is free, block 6 is occupied, and block 7 is free.
4. FJM allocates block 5 for the transaction's first data payload, logs it to its `blk_list`, and passes it back to JBD2's physical `bmap` router.
5. JBD2 requests the next payload allocation. FJM skips block 6 entirely, securely mapping the next buffer sequence directly into block 7.

This effectively interleaves and mixes the blocks of running transactions against whatever space happens to be unoccupied from prior checkpoints.

8.4 Journal space efficiency

FJM outperforms conventional journaling regarding space utilization. Because JBD2 operates rigidly, un-checkpointed transactions often create “traffic jams” within the contiguous loop. If checkpointing is slow, large sequential regions remain locked, preventing newer large transactions from finding adequate sequential space, even if *total combined* free space is plentiful overall.

FJM eliminates these spatial locks. Because allocations fall into any gap found by the bitmap, the journal utilization factor can sustainably sit at **99.9% capacity**. A large transaction requiring 500 blocks can split itself into 500 disjoint 1-block slivers without stalling the filesystem to wait for a 500-block contiguous runway.

8.5 Redundancy of credit-based reservation

Traditional ext4 depends severely on a complex **handle_t credit mechanism**. When starting a transaction, ext4 developers must heuristically “guess” the maximum worst-case contiguous blocks an operation will need (e.g., reserving 20 credits for an inode modification). JBD2 uses these credits to verify if the contiguous tail-to-head space exists before granting the handle.

With FJM, **spatial credit reservation becomes largely obsolete**.

- **Why it is not needed:** Because we no longer have to guarantee a contiguous landing zone, the risk of a single transaction breaching an arbitrary end-of-journal wrapping boundary vanishes. A block is simply a block.
- **Why it is better:** Guessing credits is notoriously inaccurate (historically leading to massive over-reservations spanning hundreds of unnecessary blocks just to be “safe”). By obsoleting rigid contiguous bounds, ext4 could dynamically allocate precisely what it needs, at the moment it needs it, checking only if the global `bitmap_weight < total_blocks`. This saves RAM overhead, speeds up the JBD2 spinlocks during transaction starts, and eliminates the dreaded JBD2: `Out of credits` panics.

8.6 Experiments

To assess the repeatability and statistical stability of the results, the full FJM benchmark suite was executed five times using the `fjm_full_benchmark.sh` script.

The experiment simulates a database Write-Ahead Log (WAL) workload using `fio`. The workload consists of 100 MB of sequential writes with an 8KB block size and one `fsync` per write (`fsync=1`). This `fsync`-heavy pattern forces frequent JBD2 journal commits, effectively stress-testing the block allocation and journal wrap-around behaviors of both standard ext4 and the FJM implementation.

The tests were performed on a dedicated 5.1 GB block device (`/dev/sdb`), formatted fresh with a 64 MB journal (`mkfs.ext4 -F -J size=64`) before each run to ensure identical starting states. The performance was evaluated across three journaling modes (`ordered`, `journal`, and `writeback`), comparing the standard ext4 implementation against the FJM-enabled module.

Mode	BW (KB/s)	IOPS	P99 (ms)	Phys (MB)	WAF
ordered_std	2674	334.3	0.409	250.0	2.50×
ordered_fjm	2827	353.5	0.053	250.0	2.50×
journal_std	2919	364.9	0.302	350.0	3.50×
journal_fjm	2301	287.7	0.424	361.3	3.61×
writeback_std	2772	346.6	0.054	250.0	2.50×
writeback_fjm	2715	339.5	0.049	250.0	2.50×

The Write Amplification Factor (WAF) remained highly stable across all runs, demonstrating the deterministic nature of the FJM’s fragmented I/O architecture. The FJM index block allocation naturally introduces a minor, predictable WAF overhead in `journal` mode (3.61× compared to 3.50×). Notably, the P99 latency significantly improved with FJM in `ordered` and `writeback` modes, indicating faster and more consistent block allocation using the in-memory bitmap search compared to standard sequential ring-buffer wrapping.

Crash recovery. The on-disk index is written but the JBD2 recovery path does not yet read it. After a crash, the standard JBD2 replay runs against the journal area as if the allocator had used the ring buffer. This is the primary item of remaining work and is documented in `shrey/README.md`.

8.7 Reproduction

```
cd /path/to/linux-6.1.4
git apply /path/to/MTech\ Rocks/shrey/patches/First.patch
git apply /path/to/MTech\ Rocks/shrey/patches/Second.patch
tar xzf /path/to/MTech\ Rocks/shrey/module/ext4_tracker.tar.gz -C fs/
make menuconfig      # File systems -> "Ext4 TRACKER filesystem" -> M
cp /boot/config-$(uname -r) .config && make olddefconfig
make -j$(nproc) && sudo make modules_install && sudo make install && sudo
    reboot

# After reboot:
sudo insmod fs/ext4_tracker/ext4_tracker.ko
cd /path/to/MTech\ Rocks/shrey/benchmarks
sudo bash fjm_full_benchmark.sh          # ~40-60 minutes
```

9 Conclusion

The four parts of the project landed independently. Milan’s characterization said the cost of journaling on a WAL workload is the cost of the commit, not the write; that pointed at the commit path and the wait-on-commit path as the targets. Suyamoon’s two patches removed two fast-commit fallback reasons, `XATTR` (inline) and `FALLOC_RANGE`, cutting JBD2 full commits by

64× and 62.5× on their respective microbenches and reproducing the architectural claim byte-identically on bare-metal. Sahil’s CAS replacement of `jbd2_log_wait_commit` removed most of the `j_state_lock` contention his `kretprobes` measurement had identified, giving 40–54% IOPS gains at 16 threads on three workloads. Shrey’s FJM module replaced the strict ring-buffer journal allocator with a free-block bitmap, cutting P99 latency on the WAL workload by 8× in `ordered` mode at the same WAF.

The shared methodological lesson is the same across all four parts: *measure the hit rate of the slow path you intend to optimize before writing kernel code*. Suyamoon’s C1 and C2 did not do that and produced no measurable improvement. Milan’s characterization, Suyamoon’s C3 and C4, Sahil’s CAS module, and Shrey’s FJM all did, and all produce results that reproduce on independent hardware.

References

- [1] A. Mathur, M. Cao, S. Bhattacharya, A. Dilger, A. Tomas, L. Vivier. *The new ext4 filesystem: current status and future plans*. Proceedings of the Linux Symposium, Ottawa, 2007.
- [2] D. Park, D. Shin. *iJournaling: Fine-Grained Journaling for Improving the Latency of Fsync System Call*. USENIX ATC 2017.
- [3] V. Chidambaram, T. S. Pillai, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau. *Optimistic Crash Consistency*. ACM SOSP 2013.
- [4] H. Shirwadkar, S. Kadekodi, T. Ts’o. *FastCommit: Resource-Efficient, Performant, and Cost-Effective File System Journaling*. USENIX ATC 2024.
- [5] Y. Oh, J. Yoo, D. Nam, C. Min, Y. Won. *CJFS: Concurrent Journaling for Better Scalability*. USENIX FAST 2023.
- [6] Q. Jiang et al. *Sync+Sync: A Covert Channel Built on fsync with Storage*. USENIX Security 2024.

A Artifact Evaluation

This appendix follows the CS614 Artifact Evaluation template.

A.1 Artifact directory structure

The submitted artifact is the ZIP `MTech Rocks.zip`, which unpacks to a single folder `MTech Rocks/` with one subdirectory per group member plus the umbrella documents at the top:

```
MTech Rocks/
|-- README.md, declaration.txt, project_report.pdf
|-- suyamoon/ (241110091): C1-C4 patches, benches, results, instructions
|-- sahil/ (241110061): jbd2_trace module, scaling matrix, results
|-- milan/ (241110042): WAL + combined fio harnesses, averaged tables
|-- shrey/ (251110068): FJM patches + ext4_tracker module + benches
+-- combined_report/ (LaTeX source of project_report.pdf)
```

The Linux kernel source tree is not in the ZIP. Reviewers should fetch the pristine 6.1.4 tarball from <https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.4.tar.xz> and apply the relevant patches to it.

A.2 Setup

Hardware. 2 cores minimum (8+ recommended for Sahil’s matrix); 4 GiB RAM minimum (8 GiB recommended); 40 GB free disk plus a dedicated ext4-formattable partition for Sahil (default `/dev/nvme0n1p7`), Milan (`nvme0n1p6`), and Shrey (`/dev/sdb`). No GPU.

Operating system. Ubuntu 22.04 or 24.04 LTS. Tested on Ubuntu 24.04.2 inside VirtualBox (Suyamoon VM data) and on a bare-metal Ubuntu host with NVMe SSD (Suyamoon bare-metal data, Sahil, Milan, Shrey).

Software dependencies.

```
sudo apt install -y fio attr build-essential libncurses-dev bison \
    flex libssl-dev libelf-dev bc dwarves zstd patch wget gawk \
    linux-tools-common linux-tools-$(uname -r) bpftrace sysbench \
    fsmark dbench python3-numpy python3-matplotlib blktrace \
    linux-headers-$(uname -r)
```

Kernel build.

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.4.tar.xz
tar xf linux-6.1.4.tar.xz && cd linux-6.1.4

# Suyamoon's two patches:
patch -p1 < /path/to/MTech\ Rocks/suyamoon/fc-inline-xattr.patch
patch -p1 < /path/to/MTech\ Rocks/suyamoon/fc-fallocate-range.patch

# Shrey's two patches + module (only needed if running C6):
git apply /path/to/MTech\ Rocks/shrey/patches/First.patch
git apply /path/to/MTech\ Rocks/shrey/patches/Second.patch
tar xzf /path/to/MTech\ Rocks/shrey/module/ext4_tracker.tar.gz -C fs/

cp /boot/config-$(uname -r) .config
sed -i 's/^CONFIG_LOCALVERSION=.*CONFIG_LOCALVERSION="-mtechrocks"/' .config
make menuconfig # for Shrey: File systems -> Ext4 TRACKER filesystem -> M
make olddefconfig
make -j$(nproc)
sudo make modules_install && sudo make install && sudo update-grub && sudo
reboot
```

Sahil’s module is built out-of-tree against any installed 6.1.4 with `linux-headers`; no in-tree patch required. Milan’s harness needs no custom kernel.

A.3 Features supported

- **Suyamoon:** C3 (inline xattr fast commit) and C4 (fallocate range fast commit). Patches under `suyamoon/`.
- **Sahil:** C5 (CAS-gated lockless `jbd2_log_wait_commit`) plus a JBD2 lock-overhead tracer. Module under `sahil/module/`.
- **Milan:** per-mode JBD2 characterization on WAL and WAL+sequential workloads. Harnesses under `milan/benchmarks/`.
- **Shrey:** C6 (Fragmented Journal Map). Patches under `shrey/patches/`; module source in `shrey/module/ext4_tracker.tar.gz`.

Findings during evaluation. No crashes, deadlocks, or assertion failures were observed during any member’s evaluation of the patched kernel. `xfstests generic/473` fails identically on stock 6.1.4 and on every patched kernel we tried; this is a pre-existing failure, not a regression. Suyamoon’s C3 and C4 patches compose cleanly. Shrey’s FJM does not yet implement the JBD2 recovery-path read of the on-disk index; documented as a known limitation.

A.4 Assumptions and unsupported

C3 covers inline xattrs only (large block-stored xattrs and `ea_inode` xattrs still fall back). C4 covers `COLLAPSE_RANGE` and `INSERT_RANGE` only; `PUNCH_HOLE` is unaffected. Sahil’s module is not measured against `data=journal` (SSD-bound). Shrey’s FJM does not yet consume the on-disk index on recovery.

A.5 Getting started (within 30 minutes)

1. Extract the ZIP. Verify Suyamoon’s patches apply cleanly (`patch -p1 -dry-run` against a pristine 6.1.4 tree).
2. Build Sahil’s module against your running kernel (`cd sahil/module && make`).
3. Inspect recorded numbers without running:

```
cat "MTech Rocks/suyamoon/eval_results_c3/baremetal/PATCHED_6.1.4-C3Patch/xattr_loop.txt"
cat "MTech Rocks/suyamoon/eval_results_c4/baremetal/PATCHED_C4_6.1.4-C3C4Patch/collapse_summary.txt"
ls "MTech Rocks/sahil/results/benchmark_results/" | head
cat "MTech Rocks/milan/benchmarks/wal_workload_no_fc/sync_heavy_bs=4k.txt" | head -30
cat "MTech Rocks/shrey/results/logs/full_benchmark1.log" | head -20
```

4. Each per-member `README.md` has the full reproduction recipe.

A.6 Detailed evaluation

The table below lists every experiment in the artifact: purpose, how to run, expected runtime, expected result, and where the output lands. The table uses `longtable` so it breaks across pages cleanly.

#	Scenario	Script	Objective / Expected outcome
<i>Suyamoon – Candidate 3: inline xattr fast commit</i>			

(continued on next page)

(continued from previous page)

#	Scenario	Script	Objective / Expected outcome
1	Primary xattr workload. 5000 fsetxattr+fsync ops; 256 MB loop fs; -O fast_commit; single inline-sized value.	suyamoon/bench_xattr.sh	Measure per-op latency and JBD2 commit count. Patched: ~23 s, tx_delta ~78. Stock: ~31 s, tx_delta 5000. 64× commit reduction; 27% wall-time reduction.
2	Non-xattr control. fio 4-job randwrite+fsync for 30 s.	suyamoon/bench_async_prefetch.sh	Confirm unrelated workloads are not regressed. Patched IOPS within ±5% of stock. Observed: 528 vs 530.
3	Crash recovery A. 100 setfattr ops on one file; lazy umount; remount; count.	suyamoon/c3_crash_test_a.sh	Expanded FC_TAG_INODE replay restores all inline xattrs. PASS with 100/100.
4	Crash recovery B. Set 100; remove 50 even-numbered; simulated crash.	suyamoon/c3_crash_test_b.sh	Removal path replays correctly via fast commit. PASS with exactly 50 odd xattrs remaining.
5	Crash recovery C. 668-byte xattr (forces the block path).	suyamoon/c3_crash_test_c.sh	Full-commit fallback is taken for block xattrs and data is preserved. PASS with all 668 bytes recovered.
<i>Suyamoon – Candidate 4: fallocate range fast commit</i>			
6	Characterization on C3 kernel. 3 microbenches (fallocate, large-xattr block, concurrent fsync).	suyamoon/bench_fallocate_range.sh, bench_xattr_block.sh, bench_concurrent_fsync.sh	Fallocate and xattr block show tx/op ≈ 1.001 (strong); concurrent fsync drops to 0.003 at T=16 (weak). Winner: fallocate.
7	Primary fallocate workload. 1000 fallocate+fsync ops for each of COLLAPSE_RANGE and INSERT_RANGE.	suyamoon/bench_fallocate_range.sh	C4 patched: tx_delta 16, ms/op 5.2–5.4. C3 baseline (arch-equivalent to stock for this path): tx_delta 1001, ms/op 7.0. 62.5× tx reduction, ~24% wall-time reduction.
8	Crash recovery A (C4). 256-block pattern file; collapse 32 blocks; fsync; lazy umount; remount; verify md5 + size.	suyamoon/c4_crash_test_a.sh	FC replay of ADD_RANGE+DEL_RANGE+FC_TAG_ID reproduces expected post-collapse state. PASS.
9	Crash recovery B (C4). Insert 16 blocks at offset 16; verify head + hole + shifted tail.	suyamoon/c4_crash_test_b.sh	PASS (md5 + size match).
10	Crash recovery C (C4). Three interleaved collapse/insert ops + crash.	suyamoon/c4_crash_test_c.sh	PASS (md5 + size match the pure-Python expected state).
<i>Suyamoon – regression gate, applies to both patches</i>			
11	xfstests subset xfstests generic/{062,118,300,337,454,455,473,482} + ext4/032.		6/6 runnable tests pass identically on stock, C3, and C4. One pre-existing generic/473 failure on all three (not our regression). Logs at suyamoon/xfstests_c3/, suyamoon/xfstests_c4/.
<i>Sahil – Candidate 5: lockless CAS for jbd2_log_wait_commit</i>			

(continued on next page)

(continued from previous page)

#	Scenario	Script	Objective / Expected outcome
12	Lock-overhead measurement. Load module with optimize=0; run fio 16-thread randwrite+fsync; rmmod prints lock-time fraction to dmesg.	sahil/benchmarks/dmesg_sampler.sh	~78% of total fsync time spent in jbd2_log_wait_commit. Recorded in sahil/results/dmesg_logs/.
13	Crash-safety smoke under the lockless module. Concurrent fsync flood + immediate umount + fsck.ext4 -n.	sahil/benchmarks/validation.sh	fsck reports clean. sahil/results/fs_log.txt.
14	Scaling matrix. ordered/writeback $\times$ opt 0/1 $\times$ threads {1, 4, 8, 16} $\times$ 3 runs $\times$ {fio, sysbench, fs_mark, dbench}.	sahil/benchmarks/run_evals.sh	+40% IOPS lock-contention, +54% IOPS metadata, +7% OLTP at 16 threads. Per-run JSON+txt under sahil/results/benchmark_results/.
<i>Milan – workload characterization</i>			
15	WAL workload across 4 modes. fio single-thread, fdatasync/write, block sizes {4, 8, 16, 32, 64} KiB, 5 runs averaged.	milan/benchmarks/wal_workload_no_fc/run_repeated.sh	Journaling halves IOPS regardless of mode; 99.6% of latency is fdatasync. Averaged tables saved as *.txt.
16	Concurrent WAL + sequential workload. Two fio jobs simultaneously bounded to the same wall time; bpftrace on JBD2 throughout.	milan/benchmarks/combined_workload_no_fc/run_repeated.sh	Sequential bandwidth mode-insensitive; WAL tail latency degrades sharply under contention. Averaged tables saved as *.txt.
<i>Shrey – Candidate 6: Fragmented Journal Map</i>			
17	FJM functional check + 6-mode benchmark. Loads ext4_tracker.ko; fresh-formats /dev/sdb (64 MB journal); runs 100 MB WAL fio across ordered/journal/writeback $\times$ {std,fjm}.	shrey/benchmarks/fjm_full_benchmark.sh	~40-60 min. ordered_fjm P99 ~0.05 ms vs ~0.4 ms std at same WAF; journal_fjm WAF ~3.61 $\times$ vs 3.50 $\times$ std (index-block cost).
18	FJM 5-run stability check.	shrey/benchmarks/run_repeated_benchmark.sh	~3.5-5 hours. WAF stable across 5 runs in every mode.